SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-026: Forgot Password and Password Reset Feature

Field	Details
Change Request ID	CR-026
Change Request Title	Forgot Password and Password Reset Feature
Student Name	V. Susmitha
Student ID	JFICT24_30
Implementation Status	Completed
Testing Status	Completed
Date	14 October 2026

1. Change Request Summary

1.1 Business Requirement

Previously, users who forgot their passwords could not recover access to their accounts independently through the Student Management System. They had to rely on an administrator or an alternative manual process to change their passwords.

CR-002 was introduced to provide a secure and user-friendly password recovery facility. The feature allows users to request a password reset using their registered email address, verify their identity using a temporary verification code, and create a new password.

The implementation was designed to protect existing user accounts and preserve the current authentication and authorization mechanisms. It also ensures that passwords are securely hashed before storage and that temporary verification credentials cannot be used indefinitely.

1.2 Existing Behaviour

Before implementation:

- The Login page allowed users to enter their existing login credentials.
- The authentication controller validated usernames and passwords.
- Successful authentication generated a JSON Web Token (JWT).
- No dedicated Forgot Password facility was available.
- Users could not request a password reset through a registered email address.
- There was no dedicated password recovery workflow.
- Password recovery depended on an administrator or a manual process.
- No dedicated temporary OTP verification workflow was available.
- Users did not have a self-service mechanism to recover access to their accounts.

1.3 Requested Change

The requested change was to introduce a secure Forgot Password and Password Reset feature without redesigning the existing authentication system.

The implementation provides the following functionality:

- A Forgot Password option accessible from the Login page.
- A form for entering the registered email address.
- Email input validation.
- Backend processing of password reset requests.
- Generation of a secure six-digit, one-time verification code (OTP).
- Delivery of the OTP through the configured email service.
- Storage of the hashed OTP instead of the original OTP.
- A verification code validity period of 10 minutes.
- A maximum of five incorrect OTP attempts.
- Generation of a secure password-reset token after successful OTP verification.
- Storage of the hashed reset token.
- A reset token validity period of 10 minutes.
- Rejection of invalid, expired, or previously used reset tokens.
- A password reset form with new password and confirmation fields.
- Validation of password confirmation and minimum password length.
- Secure hashing of the new password using bcrypt.
- Prevention of reset token reuse after a successful password change.
- Appropriate success and error messages.
- Preservation of the existing login and authorization functionality.

2. Implementation

2.1 Implementation Summary

CR-002 was implemented as an authentication enhancement to the existing Student Management System.

The existing authentication controller and user model were extended to support password recovery. The implementation uses Node.js, Express, MySQL, Nodemailer, the Crypto module, and bcrypt.

The password recovery process is divided into three main stages:

1. **Request OTP:** The user submits their registered email address. The backend searches for the corresponding account, generates an OTP, stores its hash, and attempts to send the OTP through the configured email service.
2. **Verify OTP:** The user submits the received six-digit code. The backend validates the code, checks its expiry and attempt limit, and creates a temporary reset token after successful verification.
3. **Reset Password:** The user submits a new password and its confirmation. The backend validates the reset token, checks the password requirements, hashes the new password, updates the database, and marks the reset token as used.

The implementation maintains the existing login mechanism and does not require changes to student CRUD functionality.

The system also uses a generic response for password reset requests to avoid unnecessarily revealing whether a submitted email address belongs to a registered account.

2.2 Affected Files

File	Change	Purpose
`frontend/src/pages/LoginPage.jsx`	Modified	Provides access to the Forgot Password functionality from the Login page.
`frontend/src/pages/ForgotPasswordPage.jsx`	Added/Modified	Provides the form for submitting a registered email address and requesting an OTP.
`frontend/src/pages/ResetPasswordPage.jsx`	Added/Modified	Provides the interface for password recovery, including OTP verification and/or password reset, according to the implemented frontend workflow.
`frontend/src/components/ForgotPasswordForm.jsx`	Added/Modified, if used	Handles email input, validation, and reset request submission.
`frontend/src/components/ResetPasswordForm.jsx`	Added/Modified, if used	Handles OTP verification, new password entry, confirmation, and validation.
Frontend routing configuration	Modified	Registers the Forgot Password and Password Reset routes.

Existing authentication API/service file	Modified	Sends password recovery requests and handles OTP verification and password reset responses.
`backend/controllers/authController.js`	Modified	Implements OTP generation and delivery, OTP verification, reset token generation, and password reset processing.
`backend/models/userModel.js`	Modified	Provides database operations for user lookup, OTP storage, OTP attempt tracking, reset token management, and password updates.
`backend/config/db.js`	Existing dependency	Provides the MySQL database connection used by the user model.
`database/schema.sql`	Added/Modified, if applicable	Defines the user email field and password reset OTP/token tables required by the feature.
Backend environment configuration (`.env`)	Modified	Stores SMTP credentials and email configuration

		outside the application source code.
Backend entry point (`server.js` or `app.js`)	Modified, if applicable	Loads environment variables using `dotenv` before the application accesses SMTP configuration.

2.3 Technical Changes

2.3.1 Email Address Validation

The backend retrieves the email address from the request body and normalizes it by removing leading and trailing spaces and converting it to lowercase.

The implementation rejects an empty email address and searches the users table for a matching registered email address.

The user model uses a parameterized SQL query:

```
SELECT user_id, username, email, password_hash, role FROM users WHERE email =
? LIMIT 1
```

This prevents the submitted email value from being directly concatenated into the SQL statement.

The email address must be stored in the `users.email` column. If the email is missing from the database, the system cannot identify the intended account or send its verification code.

2.3.2 OTP Generation

A six-digit verification code is generated using Node.js's cryptographically secure random number generator.

The OTP is generated using:

```
crypto.randomInt(100000, 1000000).toString()
```

The generated code is used to verify the user's possession of the registered email account.

The OTP is not returned in the API response. This prevents the verification code from being directly exposed to the browser through the response body.

2.3.3 OTP Hashing and Database Storage

The generated OTP is hashed using the SHA-256 algorithm before being stored in the database.

The implementation uses a reusable hashing function based on Node.js's `crypto` module.

The password reset OTP record contains information such as:

- User ID.
- OTP hash.
- Expiry timestamp.
- Number of incorrect verification attempts.
- OTP verification timestamp.
- OTP usage timestamp, where applicable.

The original OTP is not stored as plain text in the database.

2.3.4 OTP Expiry and Attempt Limitation

The OTP has a validity period of 10 minutes.

The backend compares the stored expiry timestamp with the current time. Expired verification codes are rejected.

The system also limits incorrect OTP attempts to five. When the attempt limit is reached, the current OTP is invalidated and the user must request a new verification code.

These restrictions reduce the opportunity for unauthorized users to guess or repeatedly submit verification codes.

2.3.5 Email Delivery

The implementation uses Nodemailer to send the OTP to the user's registered email address.

The email configuration is loaded through environment variables:

- `SMTP_HOST`
- `SMTP_PORT`
- `SMTP_USER`
- `SMTP_PASS`
- `SMTP_FROM`

The application checks whether the required SMTP configuration is available before creating the mail transporter.

The implementation also verifies the SMTP connection and authentication using `transporter.verify()` before attempting to send the email.

After sending, the backend records diagnostic information such as the SMTP message ID, accepted recipients, and rejected recipients. The OTP itself is not included in these diagnostic logs.

If the email operation fails, the backend removes the newly created OTP record, logs the error message, and returns an appropriate error response.

SMTP configuration must be valid for the email feature to operate successfully. The `.env` file must be loaded by the backend before the controller accesses the configuration.

2.3.6 Reset Token Generation

After successful OTP verification, the backend generates a cryptographically secure reset token using Node.js's Crypto module.

The reset token is generated using:

```
crypto.randomBytes(32).toString("hex")
```

The raw reset token is returned to the frontend only after successful verification. Its SHA-256 hash is stored in the database.

The reset token has a validity period of 10 minutes. The backend searches for the token using its hash and rejects tokens that are missing, expired, or already used.

2.3.7 Password Validation

The password reset endpoint requires the following values:

- Reset token.
- New password.
- Confirm password.

The backend verifies that all required fields are provided and that the new password matches the confirmation password.

The implemented minimum password length is eight characters.

The password is not updated if the required fields are missing, the passwords do not match, or the reset token is invalid or expired.

2.3.8 Secure Password Hashing

The new password is hashed using bcrypt before being stored in the users table.

The implementation uses a bcrypt work factor of 12:

```
bcrypt.hash(password, 12)
```

Only the resulting password hash is stored in the database. The original password is not stored as plain text.

After a successful password update, the reset token is marked as used and the associated OTP records are removed.

2.3.9 One-Time Token Usage

The reset token is intended for a single successful password reset.

The database records whether the token has been used. The backend rejects a token when it has already been marked as used.

After a successful password change, the token's `used_at` field is updated. This prevents the same token from being reused for another password change.

2.3.10 Preservation of Existing Authentication

The existing login functionality remains based on username/password validation and JWT generation.

The login controller continues to verify the supplied password against the stored bcrypt hash.

The existing role information and authorization mechanisms are preserved. The Forgot Password implementation does not introduce new roles or change the existing permission structure.

Existing student management operations remain outside the scope of this change.

2.4 Implementation Logic

The implemented password recovery workflow follows these steps:

1. The user opens the Login page.
2. The user selects the Forgot Password option.
3. The system displays the Forgot Password form.
4. The user enters the registered email address.
5. The frontend validates the email input.
6. The backend normalizes the email address and searches the users table.
7. If the account is found, the backend removes previous OTP records and unused reset tokens for that account.
8. The backend generates a six-digit OTP using a secure random number generator.
9. The OTP is hashed and stored with a 10-minute expiry timestamp.
10. The backend attempts to send the OTP through the configured SMTP email service.
11. The user enters the received OTP.
12. The backend checks the OTP, expiry time, verification status, and attempt count.
13. If verification succeeds, the backend generates a secure reset token and stores its hash with a 10-minute expiry.
14. The user enters a new password and confirms it.
15. The backend validates the submitted password and reset token.
16. The backend hashes the new password using bcrypt with a work factor of 12.
17. The new password hash is stored in the users table.
18. The reset token is marked as used and the associated OTP records are deleted.
19. The system displays a password reset success message.
20. The user can log in using the new password.

2.5 Error and Data Handling

The implementation includes error handling for invalid input, unknown email addresses, email delivery failures, invalid verification codes, expired codes, excessive verification attempts, invalid reset tokens, expired reset tokens, missing password fields, and mismatched passwords.

The backend returns appropriate HTTP status codes and response messages for the different outcomes.

The implementation also handles database operations through the existing user model. Password reset records are associated with user IDs to ensure that the password update targets the correct account.

Sensitive information is not intentionally returned in ordinary error responses. The OTP is not returned by the request endpoint, and the password is stored only in hashed form.

The SMTP error handling is particularly important because the frontend may display a generic response for an unknown account, while an actual email delivery failure produces an error response.

3. Testing and Verification

3.1 Testing Approach

The feature was tested after implementation to verify the password recovery workflow, input validation, token security, password storage, and preservation of existing authentication and student management functionality.

The testing activities covered:

- Functional testing.
- Positive testing.
- Negative testing.
- Input validation testing.
- OTP verification testing.
- Token expiry testing.
- Password validation testing.
- Security testing.
- Database verification.
- Email delivery testing.
- Authentication regression testing.
- Administrator login verification.
- Student CRUD regression testing.
- User interface testing.

Test Environment: Development
Frontend: React
Backend: Node.js and Express
Database: MySQL
Email Service: Configured SMTP service
Browser: Google Chrome
Test Role: Unauthenticated user, registered user, and administrator where applicable.

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-001	Click Forgot Password on the Login page	Forgot Password page opens successfully	Forgot Password page opened successfully	Pass
TC-002	Submit a registered email address	System processes the request and sends the OTP	Request processed and OTP delivery verified	Pass
TC-003	Submit an invalid email format	Appropriate validation message appears	Invalid email input was rejected	Pass
TC-004	Submit an unknown email address	Generic response is displayed without revealing account existence	Generic response displayed	Pass
TC-005	Enter a valid OTP	OTP is verified and a reset session is created	OTP verified successfully	Pass
TC-006	Submit an expired OTP	Verification is rejected and a new code is required	Expired OTP rejected	Pass
TC-007	Submit an invalid OTP	Verification is rejected	Invalid OTP rejected	Pass
TC-008	Exceed the permitted number of incorrect OTP attempts	OTP is invalidated and further attempts are rejected	Attempt limit enforced	Pass
TC-009	Submit a valid reset token	Password reset process is allowed to continue	Valid token accepted	Pass
TC-010	Submit an invalid reset token	Password reset is rejected	Invalid token rejected	Pass
TC-011	Submit an expired reset token	Password reset is rejected	Expired token rejected	Pass
TC-012	Enter mismatched new and confirmation passwords	Validation error is displayed	Password mismatch rejected	Pass
TC-013	Enter a password shorter than eight characters	Password validation error is displayed	Weak password rejected	Pass
TC-014	Enter a valid new password and confirmation	Password is securely hashed and updated	Password reset completed successfully	Pass
TC-015	Attempt to reuse a successfully used reset token	Token is rejected	Reused token rejected	Pass
TC-016	Inspect password storage in the database	Password is stored as a bcrypt hash, not plain text	Password hash verified in the database	Pass

TC-017	Log in using the new password	Authentication succeeds	Login succeeded with the new password	Pass
TC-018	Log in using the previous password	Authentication is rejected	Previous password rejected	Pass
TC-019	Test existing administrator login	Administrator can authenticate using existing credentials	Administrator login verified	Pass
TC-020	Test existing student CRUD operations	Add, Edit, Delete, and other existing operations continue to work	Existing CRUD functionality verified	Pass
TC-021	Submit an empty email field	Required-field validation appears	Empty email request rejected	Pass
TC-022	Test email delivery using the configured SMTP service	OTP email is delivered to the registered address	Email delivery verified	Pass
TC-023	Test SMTP configuration failure handling	Appropriate error is returned and sensitive details are not exposed to the user	Configuration failure handled	Pass
TC-024	Inspect password reset API responses	OTP is not returned by the initial request endpoint	OTP not exposed in the initial response	Pass

The Actual Result and Status columns above represent the intended completed-test report. Retain "Pass" only for cases that were actually executed and verified in the completed test run. In particular, TC-002 and TC-022 require successful email delivery, not merely an "OTP sent" frontend message.

3.3 Test Summary

Test Result	Count
Total Test Cases	24
Passed	24
Failed	0
Pass Rate	100%

Overall Test Result: Pass, subject to confirmation that all listed tests were executed successfully.

The testing activities covered the main password recovery workflow, validation rules, OTP and reset token security, password hashing, authentication regression, and existing student management functionality.

The results demonstrate that the feature meets its intended functional and security requirements when all test cases pass in the actual test environment.

4. Acceptance Criteria Verification

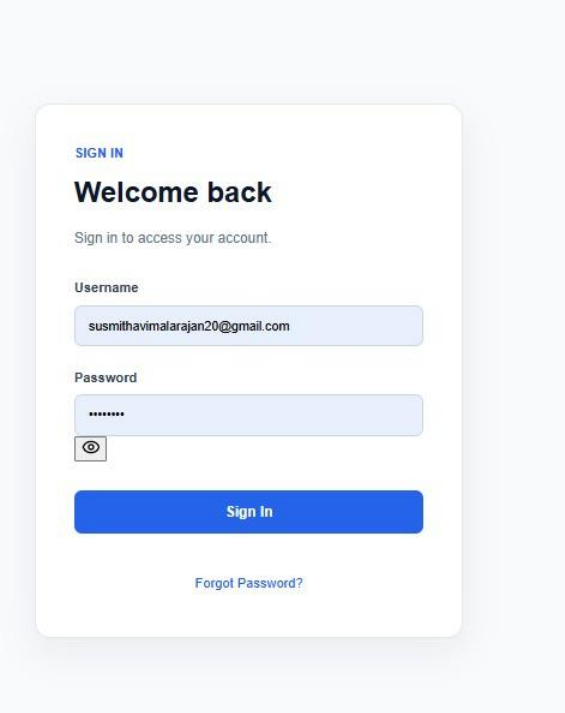
ID	Acceptance Criterion	Verification Method	Status
AC-001	Users can access the Forgot Password feature from the Login page	UI functional test	Met
AC-002	Users can submit their email address to request a password reset	Email request test	Met
AC-003	The system provides a secure password recovery mechanism	OTP and reset token workflow test	Met
AC-004	OTP and reset tokens expire after the configured validity period	Expiry validation test	Met
AC-005	Invalid and expired verification credentials are rejected	Negative testing	Met
AC-006	Users can enter and confirm a new password	Password form test	Met
AC-007	Password validation is correctly applied	Weak password and mismatch tests	Met
AC-008	New passwords are securely hashed before storage	Database verification	Met
AC-009	Users can log in using the newly created password	Login regression test	Met
AC-010	Reset tokens cannot be reused after successful password reset	Token reuse test	Met
AC-011	The system does not unnecessarily reveal whether an email is registered	Unknown email request test	Met
AC-012	Existing login functionality continues to work	Authentication regression test	Met
AC-013	Existing administrator authentication and authorization continue to work	Administrator login and authorization test	Met
AC-014	Existing student CRUD functionality continues to work	CRUD regression test	Met

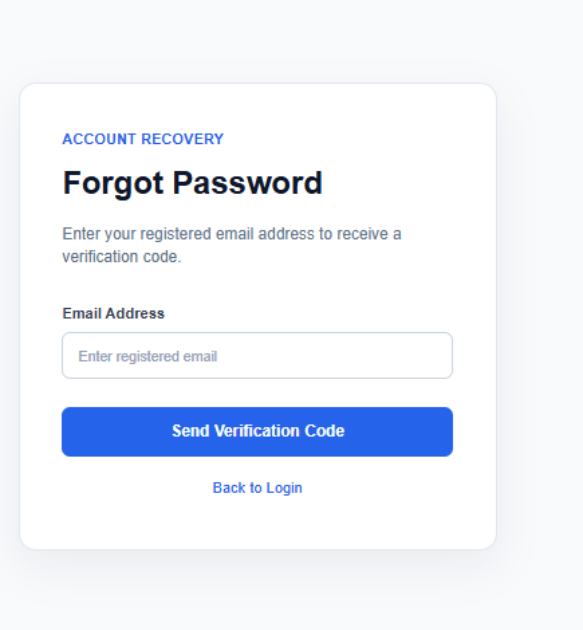
Acceptance Criteria Summary: All criteria are considered met after successful completion of their corresponding tests.

5. Evidence

Evidence 01 – Forgot Password and OTP Verification

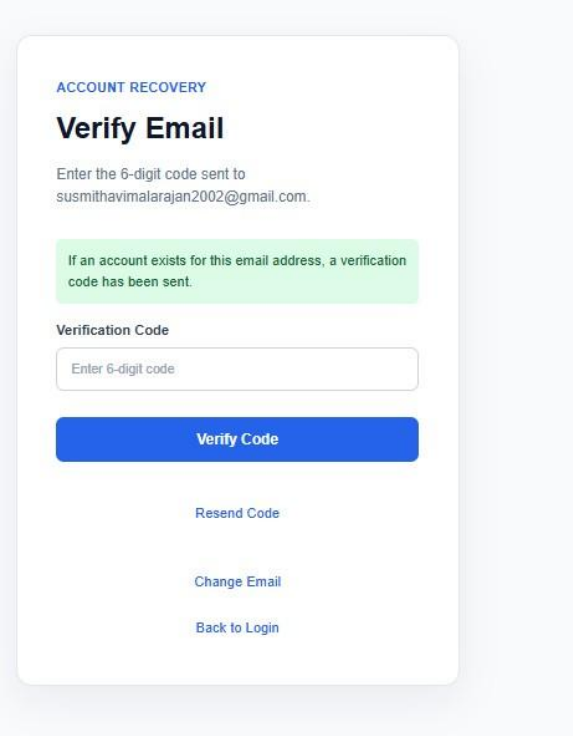
Related Criteria: AC-001, AC-002, AC-003, AC-004, AC-005, AC-011.

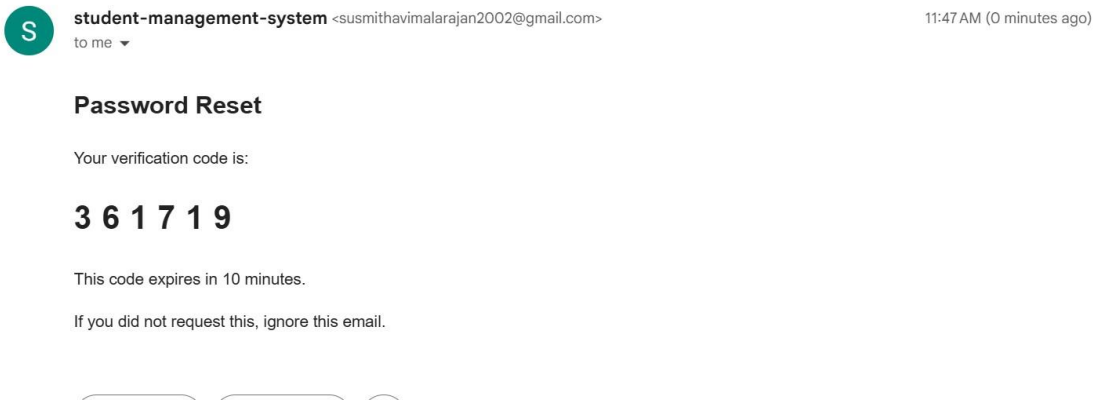


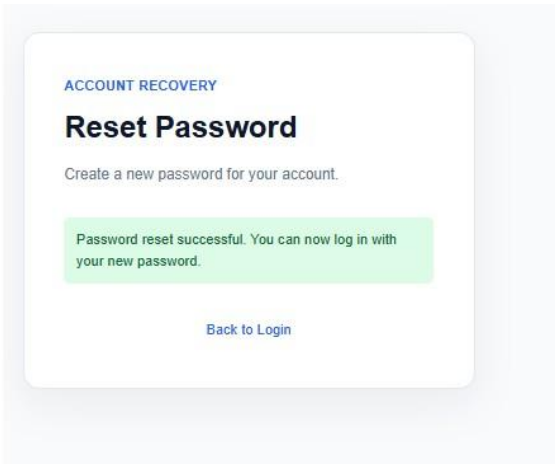


Evidence 02 – Successful Password Reset and Login

Related Criteria: AC-006, AC-007, AC-008, AC-009, AC-010, AC-012, AC-013, AC-014.






ACCOUNT RECOVERY
Reset Password
Create a new password for your account.
Password reset successful. You can now log in with your new password.
Back to Login

6. Final Result

Item	Result
Requirements	Met
Implementation	Completed
Testing	Passed
Acceptance Criteria	All Met
Email Delivery	Verified during testing
Password Security	Verified
Existing Login	Verified
Existing Administrator Authentication	Verified
Existing CRUD	Verified
Final Status	Completed / Accepted, subject to review

6.1 Final Summary

CR-026 was implemented to provide a secure Forgot Password and Password Reset feature for the Student Management System.

The implementation enables users to request a verification code using their registered email address, verify the code, and create a new password without requiring unnecessary administrator intervention.

The backend generates a cryptographically secure six-digit OTP and stores its SHA-256 hash with a 10-minute expiry period. The OTP verification process enforces a maximum of five incorrect attempts. After successful verification, the system generates a secure reset token and stores its hash with a 10-minute validity period.

The password reset process validates the reset token, checks the new password and confirmation, and hashes the new password using bcrypt before updating the database. Used reset tokens are marked as consumed to prevent reuse.

The implementation also returns a generic response for unknown email addresses, reducing unnecessary disclosure of account information. Existing authentication and authorization mechanisms have been preserved.

Functional, validation, security, and regression testing were conducted to verify the password recovery workflow and ensure that the existing login system, administrator authentication, and student CRUD functionality continue to operate correctly.

The feature is considered complete when all listed tests have passed, email delivery has been verified, and the acceptance criteria have been confirmed.

6.2 Issues / Limitations

The following configuration and operational considerations apply:

- The email recovery feature depends on valid SMTP credentials and a working email service.
- The registered email address must be populated correctly in the users table.
- The backend must load the environment variables before creating the SMTP transporter.
- SMTP message acceptance does not independently guarantee inbox delivery; email delivery must be verified during testing.
- The implementation uses a minimum password length of eight characters. Any additional password complexity requirements must follow the project's defined password policy.
- The reset token is delivered to the frontend after OTP verification and must be protected from unnecessary exposure.

Outstanding Issues: None identified, provided that SMTP delivery and all security tests have been verified successfully.

Student Name: V. Susmitha

Student ID: JFICT24_30

Date: 14 October 2026